GNoME — a parallel generative flow alongside MatterGen

uvsib

Contents

1 Motivation

MatterGen is a single generative model. Running a structurally different generator **in parallel** widens the candidate pool that feeds the (expensive, shared) ML-relax → energy-above-hull filter, at the cost of one extra cheap generation job per system. This document describes the GNoME-style generator added to the gen and `csp` paths.

The two are complementary by construction:

	MatterGen	GNoME (this flow)
Method	denoising diffusion	symmetry-aware partial substitution (SAPS)
Prior	learned from the training distribution	known crystals + data-mined ionic substitution probabilities
Bias	toward dataset-like motifs	toward prototype/Wyckoff motifs of existing crystals
Cost	GPU diffusion sampling	template seeding + cheap GNN energy screen

GNoME (Merchant *et al.*, *Nature* 2023, *Scaling deep learning for materials discovery*) found most of its stable crystals via SAPS: take a known crystal, substitute the ions on a subset of its symmetry-distinct sites with chemically plausible replacements, then filter the flood of candidates with a formation-energy GNN before DFT. This flow implements that funnel and plugs its output into the existing uvsib pipeline.

2 Is GNoME a generative ML model? (vs MatterGen)

Q: GNoME is Google DeepMind's structure-discovery method – but this flow uses no pretrained *generative* model the way MatterGen does. Is that right?

Yes, and it is faithful to GNoME by design. GNoME (Merchant *et al.*, *Nature* **619**, 80-85, 2023) and MatterGen sit in two different paradigms:

- **MatterGen** *is* a pretrained generative model: a denoising **diffusion** network that samples new crystals de novo from a learned distribution – the neural net does the generating.
- **GNoME does not generate with a neural net.** Its candidates come from two combinatorial/heuristic procedures: **SAPS** (symmetry-aware partial substitution on known templates, ranked by the data-mined ionic-substitution probabilities of Hautier *et al.* 2011) and random / symmetry-reduced structure search. GNoME's deep-learning contribution is the **filter** – an ensemble of graph neural networks trained to predict formation energy / stability that screens the flood of candidates before DFT, improved over active-learning rounds. The "deep learning for materials discovery" is that screening GNN, not a generator.

So "no pretrained generative model" is exactly the GNoME paradigm: substitution generates, a learned model filters.

Where the pretrained ML lives in this flow. There *is* a pretrained model here – in the filter role, mirroring GNoME:

stage	this flow	learned component?
generation	SAPS + MP / icet seeds (`_saps.py`, `templates.py`)	no – substitution + enumeration
stability filter	MACE single-point energy screen (`screen: MACE`)	yes – a pretrained universal MLIP

The MACE screen **stands in for GNoME's formation-energy GNN** – the one approximation versus the paper (a universal potential in place of GNoME's specific trained GNN ensemble; `--screen` can be pointed at a real GNoME checkpoint if one is wired in). In short: **GNoME = rule-based generation + a learned stability filter; MatterGen = a learned generative model** – which is why there is no generative network in the GNoME branch.

3 Where it plugs in

The generator emits the **same `output.json` contract as MatterGen** (a list of pymatgen structure dicts), so nothing downstream changes. It is submitted *beside* MatterGen and merged before the shared ML relaxation:

```
gen:  GeneratorWorkChain
        ├── mattergen.base ─┐
      └── gnome.base ──────┤→ merge structures → ML relax → unique_low_energy_chemsys → store "gen

csp:  CSPWorkChain
        ├── mattergen.csp ─┐
            └── gnome.csp ──────┤→ extend csp_structures → ML relax → minima-
hopping → store "csp"
```

Both generators are independently toggled in `input.yaml` (`mattergen.enabled`, `gnome.enabled`) and **at least one must be on**. Whichever are enabled run in parallel and are merged before the shared ML relaxation. Each is **best-effort**: a failed or empty branch is logged and the run continues on the other generator's structures; a system fails only if *no* generator yields any structure. (Historically MatterGen was mandatory and GNoME best-effort; the `mattergen` switch made the two symmetric.)

4 Components

```
codes/gnome/
  calculation.py  GNoMECalculation — stages the runner + sidecars + templates.json, retrieves ou
  parser.py      GNoMEParser      — output.json -> Dict(structures=[...]) (same as MatterGen)
    workchain.py          GNoMEBaseWorkChain (gen), GNoMECSPWorkChain (csp)    —
BaseRestart, build seeds + cmdline
    templates.py          build_template_pool()    — LOCAL: seeds from MP same-
space + icet alloys + analog chemistries; bundled fallback
codes/files/
    gnome_generate.py    runner (-> aiida.py): SAPS generate -> charge/dedup -
> optional GNN screen -> output.json
  _saps.py           symmetry-aware partial substitution (pymatgen only)
```

Entry points: `gnome` (calc), `gnome_parser`, `gnome.base`, `gnome.csp`.

4.1 SAPS generation (`_saps.py`)

- **Targets.** `gen` takes a chemical system (`Y-Ru-O`) and keeps **every** common oxidation state of each element as a candidate species — that is what lets a template Ti4+ site be recognised as a donor for Ru4+ while a Na+ site is a donor for Y3+. `csp` takes a formula (`Y2Ru2O7`) and oxidation-decorates it.
- **Donors.** For each target species, the data-mined ionic-substitution table (`SubstitutionPredictor`, Hautier *et al.* 2011) ranks the donor species it can substitute *for*. Substitutions are kept anion→anion / cation→cation, and charge-preserving swaps are favoured (they keep the cell neutral).
- **Symmetry-aware & partial.** Templates are reduced to their Wyckoff orbits (`SpacegroupAnalyzer`). In `gen`, orbits whose element is not yet in the target system are substituted (mandatory), and subsets of the remaining in-system orbits may also be swapped (partial, up to `partial` orbits) — one prototype thus seeds several off-stoichiometry candidates. In `csp`, orbits are mapped prototype→target so the product reduces **exactly** to the requested formula (prototype-substitution CSP); only templates whose anonymous formula matches the target are used.

4.2 Template seeding (`templates.py`, local side)

The seed pool is merged from three complementary sources (priority order: most-relevant first), deduplicated, then capped at `seed_cap`:

1. **Same chemical space (MP).** Real structures for the target system *and every subsystem* (elements, binaries, …) pulled from the Materials Project (stable, `e_above_hull` ≤ `seed_ehull`). For `Cu3Au` this brings in `Cu`, `Au`, and the actual Cu–Au compounds (`Cu3Au`, `CuAu`, …) — so SAPS seeds on, and emits, the real alloys instead of only elemental cells.
2. **icet-enumerated ordered alloys** (`use_icet`/`icet_seeds`, default on). Symmetry-inequivalent decorations of metallic parent lattices (fcc/bcc) with the system's elements, up to `icet_max_size` atoms (Hart–Forcade enumeration via icet). `csp` `Cu3Au` → the fcc ($L1_2$) and bcc Cu3Au orderings; `gen` `Cu-Au` → the full ladder Cu, Au, CuAu ($L1_0$), Cu2Au, CuAu2, Cu3Au, CuAu3 across fcc/bcc. Requires icet in the daemon environment; skipped silently (MP-only) if absent.

3. **Analog chemistries (MP).** The original GNoME-style donor seeding: for each target element the data-mined table gives the top-k_donors donor elements; swapping one element at a time yields analog systems pulled from MP:
   - gen Y-Ru-O → O-Ti-Y, O-Sc-Y, Mn-O-Y, O-Re-Ru, … plus Y-Ru-O.
   - csp Y2Ru2O7 → Y2Ti2O7, Na2Ru2O7, Re2Ru2O7, … ($A_2B_2O_7$ prototypes).

If all MP sources are unreachable the builder falls back to the bundled r2SCAN entries, so the branch never hard-fails.

4.3 GNN screen (gnome_generate.py)

After charge-neutrality and primitive-cell de-duplication (reusing refine.py), candidates are optionally ranked by an MLIP single-point energy and the lowest energy-per-atom per composition are kept (--screen, default MACE). This is the cheap pre-DFT filter that stands in for GNoME's formation-energy GNN; set screen: none to skip it (e.g. a CPU node), or point it at the real GNoME checkpoint once a make_calculator branch is added for it. The downstream unique_low_energy_* step is still the final stability arbiter.

5 Configuration

Opt-in; **off by default** so existing runs are unaffected.

input.yaml:

```yaml
gnome:
  enabled: true          # flip on after registering the GNoME@v100 code
GNoME_generate:
  n_max: 200             # cap on SAPS candidates before screening
  max_per_template: 8
  threshold: 0.0001      # min substitution probability
  partial: 2             # max orbits swapped at once
  keep: 60               # kept after the GNN screen
  screen: MACE           # MLIP tag, or 'none'
  head: omat_pbe         # MLIP task head for the screen (see HEADS in input.yaml)
  k_donors: 3            # donor elements per site when seeding from MP
  seed_ehull: 0.10
  seed_cap: 60
GNoME_CSP:
  num_runs: 1
  ...                    # same keys
```

config.yaml:

```yaml
codes:
  GNoME:
    code_string: GNoME@v100   # an env with pymatgen + the screen MLIP; run_aiida_python wrap
    job_script: { device: cuda, nodes: 1, ntasks: 1, cpus: 12, ngpu: 1, time: 43200, exclusiv
```

5.1 Parameter reference

All keys live under GNoME_generate and GNoME_CSP in input.yaml (both blocks take the same keys; partial is gen-only, num_runs is csp-only). Listed in funnel order.

Parameter	Funnel stage	What it controls	Default
`gnome.enabled`	—	run the GNoME generator at all (paired with `mattergen.enabled`; ≥1 required)	`false`
`k_donors`	template seeding	donor elements per target element → analog chemistries pulled from MP	`3`
`seed_ehull`	template seeding	max MP `e_above_hull` (eV/atom) for a seed prototype	`0.10`
`seed_cap`	template seeding	hard cap on seed prototypes (lowest-hull kept first)	`60`
`icet_seeds`	template seeding	also seed with icet-enumerated ordered alloys for the formula/space	`true`
`icet_max_size`	template seeding	max atoms/cell for icet enumeration (auto-raised to reach a csp formula)	`4`
`max_per_template`	SAPS	candidates kept per prototype (top-scored by substitution probability)	`8`
`partial` *(gen)*	SAPS	max symmetry orbits swapped simultaneously → off-stoichiometry spread	`2`
`threshold`	SAPS	min data-mined substitution probability to consider a donor	`1e-4`
`n_max`	SAPS	global cap on candidates before the screen	`200`
`screen`	GNN screen	MLIP tag for the energy screen (`MACE`/`uPET`/`UMA`/`MatterSim`) or none	`none`
`head`	GNN screen	MLIP task head for the screen model (see HEADS in `input.yaml`)	—
`keep`	GNN screen	**hard cap on returned structures**; also sets polymorphs kept per composition	`60`
`num_runs` *(csp)*	—	number of independent GNoME CSP jobs	`1`

The funnel, with the knob that governs each stage:

```
build_template_pool(seed_cap, k_donors, seed_ehull)        <- structural variety is born here
  -> saps_generate(max_per_template, partial, threshold, n_max) <- compositional variety
      -> charge-neutral + primitive-cell dedup
        -> screen_by_energy(keep)                            <- hard output cap + per-
composition cull
         -> [downstream, shared] ML-relax -> store only e_above_hull <= EHULL_ML (0.05) & structure
unique
```

5.2 Tuning: more structures, more variety

SAPS only re-decorates known template lattices — it never invents a new lattice. GNoME's structural variety is therefore a strict subset of the template pool's variety. For genuinely new prototypes you need MatterGen (de-novo, `mattergen.enabled: true`) or the downstream MinimaHopping step; to get more of, and more spread across, the known prototypes, tune the knobs below. The two goals use different knobs.

5.2.1 More raw structures

Output ≈ `min(keep, distinct survivors)`. Today the run is starved well before keep, since the raw candidate count ≈ `seed_cap × max_per_template`. So raise the pool **and** the cap together — bumping keep alone does nothing while the pool is the bottleneck.

knob	typical → aggressive	effect
`keep`	60 → 400–600	hard cap on returned structures; nothing else matters if this stays low
`max_per_template`	8 → 30–50	variants kept per prototype (8 discards the more exotic ones)
`seed_cap`	60 → 200–400	number of prototypes → more candidates *and* more variety
`n_max`	200 → 10000	global pre-screen cap; raise so it stops being the ceiling

5.2.2 More variety

- **`seed_cap` (biggest lever).** `_mp_by_chemsys` sorts analog structures by `e_above_hull` and truncates to the cap, so a low cap discards exactly the more metastable, structurally diverse parents.
- **`seed_ehull`** (↑, e.g. 0.10 → 0.25) — admits more metastable prototypes.
- **`k_donors`** (↑, e.g. 3 → 7–10) — more analog chemistries per element seed.
- **`partial`** *(gen, 2 → 3)* — more simultaneous orbit swaps → off-stoichiometry / solid-solution spread. Combinatorial in the number of *optional in-system orbits*, so it does little for small cells with few orbits.
- **`threshold`** (↓, 1e-4 → 1e-5) — admits lower-probability, more exotic substitutions (modest extra variety, slight junk risk).

5.2.3 Two hidden culls to be aware of

1. **The screen dedups per composition.** It keeps roughly `per_comp = keep // n_compositions` lowest-energy structures per composition, so a small keep keeps only the lowest-energy polymorph or two and throws away the metastable ones — exactly the polymorphs a synthesizability study wants. Raising keep restores polymorph variety, not just total count. (With `screen: none`, keep is **not** applied — output is everything that is charge-neutral and unique.)

2. **The downstream storage gate.** GNoME candidates are ML-relaxed and then kept only if `e_above_hull <= EHULL_ML` (0.05 eV/atom, `settings.py`) and structure-unique. The final stored count is bounded by *distinct near-hull* structures, not by keep. To retain phases above 0.05 raise `EHULL_ML` — but it is global (it also affects MatterGen and the synthesizability `meta_window`), so change it deliberately.

5.2.4 Cost

The only GPU work is the screen — one MLIP single-point per surviving candidate, linear in count. Single points are cheap, so going from a few hundred to several thousand candidates stays in the minutes range; the MP seeding queries (network, $\propto$ `k_donors × seed_cap`) are the other time sink.

5.2.5 A concrete "crank it up" profile

Apply to both `GNoME_generate` and `GNoME_CSP`:

```yaml
n_max: 10000
max_per_template: 40
threshold: 0.00001
partial: 3            # gen only
keep: 500
screen: MACE
head: omat_pbe
k_donors: 7
seed_ehull: 0.25      # 0.30 for csp
seed_cap: 300
```

5.3 Turning it on

1. Register an AiiDA code `GNoME@v100` whose executable is the `run_aiida_python` wrapper (`python -u aiida.py "$@"`) in an environment with pymatgen, mp-api and the screen MLIP (e.g. MACE). No separate generative binary is needed — the runner does the work.
2. Set `gnome.enabled: true` in `input.yaml`.

6 Status / validation

Verified offline (CPU, `screen: none`): SAPS turns a rutile template into Ru02 and an $A_2B_2O_7$ prototype into Y2Ru207; `csp` recovers the `Fd-3m` pyrochlore from a Y2Ti207 prototype; the runner produces a valid `output.json`. The AiiDA submission, the MP seeding queries and the GPU GNN screen run on the HPC and are exercised there.